7.2.2. Stopping those pesky leaks: managing memory in lock-free data structures

When we first looked at `pop()`, we opted to leak nodes in order to avoid the race condition where one thread deletes a node while another thread still holds a pointer to it that it's just about to dereference. However, leaking memory isn't acceptable in any sensible C++ program, so we have to do something about that. Now it's time to look at the problem and work out a solution.

The basic problem is that you want to free a node, but you can't do so until you're sure there are no other threads that still hold pointers to it. If only one thread ever calls `pop()` on a particular stack instance, you're home free. `push()` doesn't touch the node once it's been added to the stack, so the thread that called `pop()` must be the only thread that can touch the node, and it can safely delete it.

On the other hand, if you need to handle multiple threads calling `pop()` on the same stack instance, you need some way to track when it's safe to delete a node. This essentially means you need to write a special-purpose garbage collector just for `node`s. Now, this might sound scary, but although it's certainly tricky, it's not *that* bad: you're only checking for `node`s, and you're only checking for nodes accessed from `pop()`. You're not worried about nodes in `push()`, because they're only accessible from one thread until they're on the stack, whereas multiple threads might be accessing the same node in `pop()`.

If there are no threads calling `pop()`, it's perfectly safe to delete all the nodes currently awaiting deletion. Therefore, if you add the nodes to a "to be deleted" list when you've extracted the data, then you can delete them all when there are no threads calling `pop()`. How do you know there aren't any threads calling `pop()`? Simple—count them. If you increment a counter on entry and decrement that counter on exit, it's safe to delete the nodes from the "to be deleted" list when the counter is zero. Of course, it will have to be an atomic counter so it can safely be accessed from multiple